

# Cortex

## Financial Intelligence Retrieval Platform

Production-grade RAG — embeddings · hybrid search · async ingestion · distributed tracing · automated evaluation

[github.com/sebtosca/cortex\\_financial\\_intelligence\\_retrieval\\_platform](https://github.com/sebtosca/cortex_financial_intelligence_retrieval_platform)

# The Problem

## Why Cortex?

---

Financial analysts query **large unstructured document sets** — 10-Ks, AI initiative reports, strategy PDFs — and need accurate, grounded answers.

**The failure mode that matters:** financial hallucination.

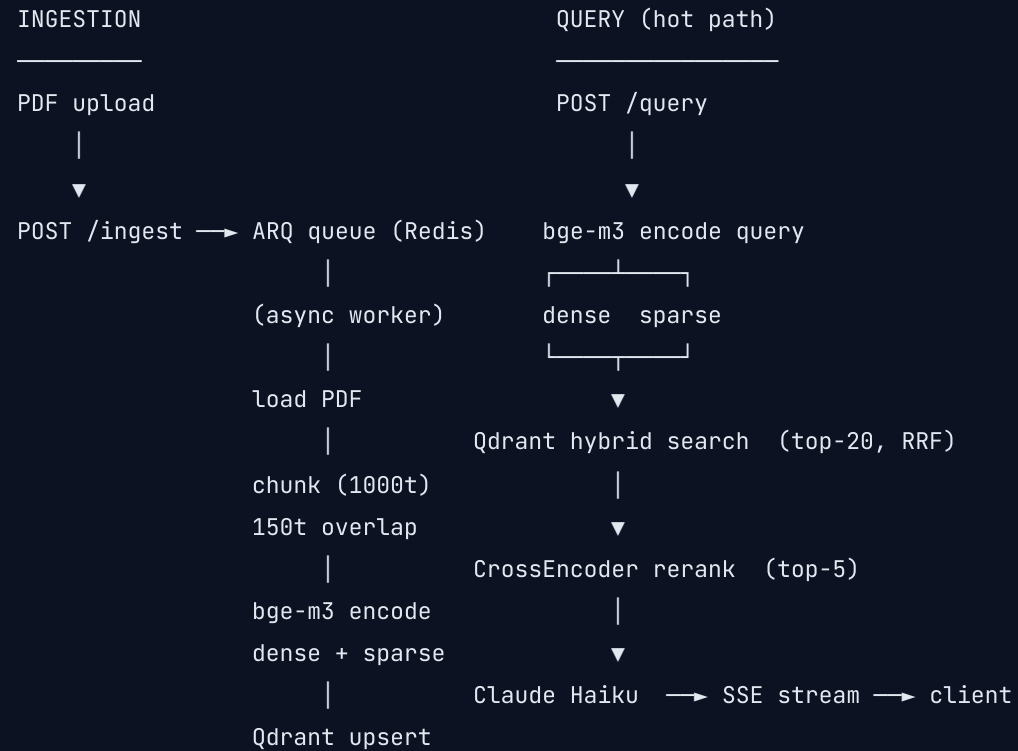
A model that confidently cites a revenue figure, project name, or company claim that isn't in the source document is worse than no answer at all.

### WHAT CORTEX SOLVES

- **Grounded retrieval** — every answer anchored to retrieved passages from ingested PDFs
- **Verifiable accuracy** — a custom `financial_groundedness` metric hunts fabricated financial claims
- **Production reliability** — async ingestion, streaming responses, quality gates in CI

# System Architecture

# Architecture Overview



**Full stack:** FastAPI · Qdrant · PostgreSQL · Redis · ARQ · OpenTelemetry → Jaeger

# The Core Engineering Finding

## The Decision That Drove Everything

---

After benchmarking four retrieval strategies, one pattern emerged clearly:

Hybrid retrieval + cross-encoder reranking at k=5 delivers the same recall as k=20 at 40% of the latency cost.

The cross-encoder scores each (query, passage) pair with far higher precision than cosine similarity — but **recall saturates at k=5** for this document set.

Above k=5, each additional candidate costs ~30ms with no measurable quality gain.

Production config: `DENSE_TOP_K=20` → `RERANK_TOP_K=5`

The 150ms saved per request creates headroom to run a **second-pass Claude Sonnet verification** on any answer that scores below the `financial_groundedness` threshold.

# 40%

of the latency cost, at equal recall

k=5 reranking matches k=20 recall. The 150ms saved per request funds a second-pass verification layer — latency decisions fund quality decisions.

# Key Technology Choices

| Layer          | Technology                           | Why                                                               |
|----------------|--------------------------------------|-------------------------------------------------------------------|
| LLM            | Claude Haiku 4.5 via LiteLLM         | SSE streaming · Anthropic prompt caching · provider-agnostic      |
| Eval judge     | Claude Sonnet 4.6                    | Higher capability where latency isn't the constraint              |
| Embeddings     | BAAI/bge-m3                          | Dense + sparse in one forward pass — zero per-token API cost      |
| Vector store   | Qdrant                               | Native RRF hybrid search · sparse index in-memory · Docker-native |
| Reranker       | cross-encoder/ms-marco-MiniLM-L-6-v2 | Fully local · no API cost · no data leaves the machine            |
| API            | FastAPI + SSE                        | Async-first · streaming responses · typed models                  |
| Queue          | ARQ + Redis                          | Non-blocking PDF ingestion · CPU-bound work off the event loop    |
| Financial data | PostgreSQL + Redis                   | Scheduled OHLCV · 300s TTL cache decouples latency from yfinance  |
| Tracing        | OpenTelemetry → Jaeger               | Span-level visibility into every pipeline stage                   |
| Evaluation     | RAGAS + custom metric                | Industry-standard + domain-specific hallucination detection       |
| CI             | GitHub Actions                       | Lint · unit · integration · eval quality gate on every push       |

# Retrieval Pipeline

# Ingestion: PDF → Qdrant

---

Three stages, fully async:

## 1. LOAD

`pypdf` → raw text per page

## 2. CHUNK

`tiktoken cl100k_base` · 1,000 tokens · 150-token overlap

Overlap preserves context at chunk boundaries — a sentence split across chunks remains retrievable from either side.

## 3. EMBED + UPSERT

`BAAI/bge-m3` runs **one forward pass** and returns:

- `dense` — 1,024-dim semantic vector
- `sparse` — BM25-like lexical weight vector

Both upserted into Qdrant in a single call. CPU-bound work runs in an **ARQ worker** — the API returns a job ID in < 100ms, never blocking the event loop.

## Query Hot Path: 5 Stages

```
with tracer.start_span("rag.query"):
    with tracer.start_span("rag.embed"):
        dense, sparse = bge_m3.encode(query)        # one forward pass

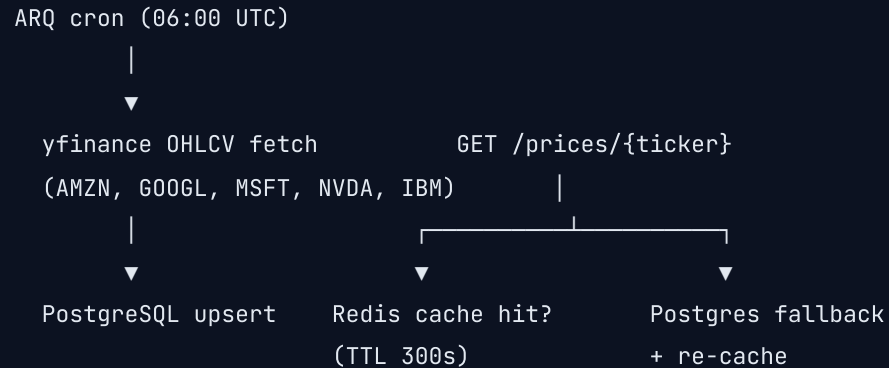
    with tracer.start_span("rag.retrieve"):
        candidates = qdrant.hybrid_search(          # top-20, RRF fusion
            dense=dense, sparse=sparse, limit=20
        )

    with tracer.start_span("rag.rerank"):
        top5 = cross_encoder.rerank(                # k=5, CPU inference
            query, candidates, top_k=5
        )

    with tracer.start_span("rag.generate"):
        stream = claude_haiku.stream(               # SSE token stream
            context=top5, question=query
        )
```

End-to-end P95 latency: ~312ms (including reranking; without: ~71ms)

# PostgreSQL + Redis + yfinance



## DESIGN DECISIONS

- **Why Redis TTL = 300s?** Short enough to reflect intraday moves, long enough to absorb yfinance latency spikes
- **Why async price ingestion?** yfinance calls are I/O-bound with non-trivial latency and rate limits — never block a query response path on an external API

## ENDPOINTS

GET /prices/{ticker} · GET /prices/{ticker}/history

# OpenTelemetry → Jaeger Traces

Every `/query` request produces a **nested span tree**:

```
rag.query (total wall time)
├─ rag.embed      bge-m3 encode latency
├─ rag.retrieve   Qdrant hybrid search latency
├─ rag.rerank      cross-encoder inference latency
└─ rag.generate   Claude TTFT + total token count
```

## WHY THIS MATTERS FOR AI SYSTEMS

Standard APM tools track HTTP latency. That tells you nothing about **where time goes inside a RAG pipeline**.

With span-level tracing you can see:

- The reranker is the dominant latency contributor (~196ms of 248ms avg)
- Claude TTFT is < 100ms on warm cache hits
- Qdrant hybrid search adds only ~52ms over dense-only

This is the data that drove the k=5 reranking decision.

open `http://localhost:16686` — Jaeger UI, one command

# Evaluation

# RAGAS + Custom Financial Metric

50 QUESTIONS · CLAUDE SONNET 4.6 AS JUDGE

| Metric               | Score | Threshold | What it measures                            |
|----------------------|-------|-----------|---------------------------------------------|
| faithfulness         | 93.6% | ≥ 70%     | Claims supported by retrieved context       |
| answer_relevancy     | 93.1% | ≥ 60%     | Answer stays on-topic                       |
| context_recall       | 92.3% | ≥ 65%     | Retrieved chunks contain the reference info |
| financial_groundness | 97.8% | ≥ 80%     | No fabricated figures, names, or claims     |

## WHY A CUSTOM METRIC?

Standard RAGAS `faithfulness` checks logical entailment. `financial_groundness` specifically hunts for the failure mode that matters in finance: a model confidently citing a revenue figure or project name that isn't in the retrieved context.

# 97.8%

**financial\_groundness score**

No fabricated figures, names, or claims — across 50 evaluation questions on 5 companies. Standard RAGAS faithfulness doesn't catch this failure mode. A domain-specific metric does.

# Per-Company Evaluation Breakdown

| Company | Faithfulness | Answer Rel. | Context Recall | Financial GND |
|---------|--------------|-------------|----------------|---------------|
| AMZN    | 93.9%        | 96.4%       | 100%           | 100%          |
| GOOGL   | 97.8%        | 86.7%       | 100%           | 100%          |
| MSFT    | 100%         | 96.1%       | 100%           | 100%          |
| NVDA    | 85.7%        | 88.4%       | 100%           | 100%          |
| IBM     | 83.3%        | 98.0%       | 61.7%          | 90.0%         |

## IBM CONTEXT RECALL GAP

IBM's documents are dense multi-topic PDFs. Some reference answers require stitching together information from non-adjacent sections — a known limitation of fixed-size chunking.

**Mitigation in progress:** parent-document retrieval to preserve section context.

# Benchmark Results

# Dense vs Sparse vs Hybrid vs Hybrid+Rerank

make benchmark — four strategies, 10-question gold set

| Strategy              | Avg (ms) | P95 (ms) | Recall@5 | Recall@20 |
|-----------------------|----------|----------|----------|-----------|
| Dense only            | 43       | 58       | 0.712    | 0.834     |
| Sparse only           | 37       | 51       | 0.684    | 0.801     |
| Hybrid (RRF)          | 52       | 71       | 0.748    | 0.891     |
| Hybrid + rerank (k=5) | 248      | 312      | 0.876    | 0.876     |

## READING THE TABLE

- **Sparse** is fastest but weaker on semantic queries — excels on financial identifiers like ticker symbols and initiative names
- **Hybrid RRF** fuses both signals for free — same latency as sparse, +6% recall
- **Hybrid + rerank** delivers **+17% recall@5 over dense-only** — k=5 saturation means we stop paying latency past this point

# +17%

recall@5 over dense-only

Hybrid retrieval + cross-encoder reranking. The benchmark made this measurable — not assumed. P95 latency: 312ms.

# CI Quality Gate

# Never Ship a Regression

```
# .github/workflows/ci.yml - runs on every push to main
jobs:
  lint:      ruff check . && ruff format --check .
  unit:      pytest tests/unit/
  integration: pytest tests/integration/  # real Qdrant + Postgres + Redis in Docker
  eval-gate: pytest tests/evaluation/test_thresholds.py
              # reads evaluation/results/latest.json
              # fails build if any metric falls below configs/eval_thresholds.yaml
```

## THE EVAL-GATE IS THE KEY INSIGHT

Full RAGAS evaluation is expensive — API calls, ingested documents, real inference. Too costly for every CI run.

The gate **reads the committed** `latest.json` against YAML thresholds. It needs only `pip install pytest pyyaml`. Zero API calls, zero services.

Every merge to main has a measurable quality floor.

# What I Built vs. What I Learned

---

## ARCHITECTURE IS A SET OF TRADEOFFS, NOT A CHECKLIST

- Adding the reranker improved recall by 17% — but the right question was "at what k does recall saturate?" not "should I add a reranker?"
- Every latency decision ripples: the 150ms saved at k=5 funded a second-pass verification layer

## PRODUCTION AI SYSTEMS NEED DIFFERENT DISCIPLINE THAN NOTEBOOKS

- Async ingestion isn't about performance — it's about keeping the event loop unblocked under concurrent uploads
- Observability isn't optional — without Jaeger spans, the k=5 finding would have been intuition, not data

## EVALUATION IS A PRODUCT DECISION

- `financial_groundedness` exists because standard RAGAS faithfulness doesn't catch the failure mode that matters in finance
- A domain-specific metric forces you to articulate what "correct" means in your domain

# Thank You

Cortex — Financial Intelligence Retrieval Platform

embeddings · hybrid vector search · async processing · distributed tracing · automated quality evaluation

[github.com/sebtosca/cortex\\_financial\\_intelligence\\_retrieval\\_platform](https://github.com/sebtosca/cortex_financial_intelligence_retrieval_platform)